



Práctica 2 - PDDL

Robótica Cognitiva - Curso 2024/2025
Máster en Robótica y Sistemas Inteligentes

Resumen

Esta práctica introduce los conceptos básicos de la Planificación Automática en Robótica mediante el uso de PDDL (Planning Domain Definition Language). Se trabajará con definiciones de dominios y problemas, explorando cómo generar planes y ejecutarlos sobre entornos simulados o representaciones simples.

■ Objetivos:

- Comprender la estructura de un problema y un dominio en PDDL.
- Definir acciones planificables con precondiciones y efectos.
- Resolver problemas de planificación mediante planners externos (popf, etc.).
- Explorar la ejecución de planes en entornos robóticos simples.

■ Número de sesiones en el aula:

- 4 sesiones

■ Fecha de entrega límite:

- 30 de mayo

■ Mecanismo de entrega (dual):

1. Subir a Moodle un documento con las respuestas y capturas (PDF preferiblemente).
2. Repositorio con el código y los dominios/problemas en: <https://github.com/fjrodl/PDDL-course>

Requisitos

Será necesario disponer de un entorno con soporte para planners en PDDL. Se recomienda:

- Ubuntu 22.04
- Python 3.10+
- Tienes varios planners en el repositorio (ver README del repositorio)

Repositorio con el material base y ejemplos: <https://github.com/fjrodl/PDDL-course>

Ejercicios (sesiones 1-3)

1. Análisis de dominio simple

- Revisa el dominio y el problema asociado de los ejercicios 1 y 2 del repositorio.
- Explica las acciones disponibles y su utilidad para alcanzar el objetivo del plan.

2. Extensión del dominio

- Añade una nueva acción al dominio que quieras del repositorio .
- Ajusta el problema para requerir el uso de esta nueva acción y genera un nuevo plan.

3. Integración de PDDL en entorno simulado

- Utiliza alguno de los planificadores del repositorio para visualizar la ejecución del plan paso a paso.
- Comenta cómo se corresponde cada acción PDDL con un comportamiento observable en el entorno.

4. Aplicación final: navegación entre waypoints usando planificación PDDL

- Crea un dominio y un problema PDDL que represente un robot que debe navegar entre varios puntos de interés.
- El plan debe incluir al menos 3 movimientos .
- Ejecuta el plan en un entorno (puede ser simulado en texto o visual con RViz/Gazebo si lo integras).
- Entrega:
 - Archivos .pddl (dominio y problema).
 - Capturas del plan generado.
 - Un vídeo (opcional) de 2–4 minutos explicando la planificación y la ejecución. Añade el enlace en el README del repositorio.
- Extra: Define otras condiciones como: obstáculos, estados del robot (cargado, descargado, portando objeto, etc.), y/ prioridades en los puntos de visita.

Ejemplo básico de planificación con PlanSys2 (Sesión 4)

Este apéndice muestra cómo instalar y probar PlanSys2 en ROS 2 Humble, usando un ejemplo básico en el que un robot debe transportar un objeto desde una sala a otra. El ejemplo está inspirado en los primeros ejercicios de introducción a PDDL.

1. Instalación de PlanSys2

```
1 # Crear un workspace
2 mkdir -p ~/plansys2_ws/src
3 cd ~/plansys2_ws/src
4
5 # Clonar PlanSys2
6 git clone --branch humble https://github.com/IntelligentRoboticsLabs/ros2_planning_system.
  git
7
8 # Instalar dependencias
9 cd ~/plansys2_ws
10 rosdep install --from-paths src --ignore-src -r -y
11
12 # Compilar
13 colcon build --symlink-install
14 source install/setup.bash
```

2. Ejemplo de dominio y problema PDDL

Archivo: robot_domain.pddl

```
1 (define (domain robot_transport)
2   (:requirements :strips :typing)
3   (:types location object)
4
5   (:predicates
6     (robot_at ?l - location)
7     (connected ?from ?to - location)
8     (object_at ?o - object ?l - location)
9     (holding ?o - object)
10    (hand_empty)
11  )
12
13  (:action move
14    :parameters (?from ?to - location)
15    :precondition (and (robot_at ?from) (connected ?from ?to))
16    :effect (and (not (robot_at ?from)) (robot_at ?to))
17  )
18
19  (:action pick
20    :parameters (?o - object ?l - location)
21    :precondition (and (robot_at ?l) (object_at ?o ?l) (hand_empty))
22    :effect (and (not (object_at ?o ?l)) (not (hand_empty)) (holding ?o))
23  )
24
25  (:action place
26    :parameters (?o - object ?l - location)
27    :precondition (and (robot_at ?l) (holding ?o))
28    :effect (and (object_at ?o ?l) (hand_empty) (not (holding ?o)))
29  )
30 )
```

Archivo: robot_problem.pddl

```
1 (define (problem robot_problem)
2   (:domain robot_transport)
3   (:objects
4     room1 room2 - location
5     box - object
6   )
7   (:init
8     (robot_at room1)
9     (object_at box room1)
10    (connected room1 room2)
11    (connected room2 room1)
12    (hand_empty)
13  )
14  (:goal
15    (and (object_at box room2))
16  )
17 )
```

3. Lanzamiento de PlanSys2

1. Lanzar el sistema con el modelo:

```
1 ros2 launch plansys2_bringup plansys2_bringup_launch_distributed.launch.py \
2   model_file:=/ruta/completa/robot_domain.pddl
```

2. Lanzar el nodo de ejecución:

```
1 ros2 run plansys2_executor plansys2_executor_node
```

3. Ejecutar el terminal de PlanSys2:

```
1 ros2 run plansys2_terminal plansys2_terminal_node
```

4. Cargar el problema y generar el plan:

- Utilizando el terminal
- Utilizando un programa Python

4. Resultado esperado

PlanSys2 generará un plan similar al siguiente:

```
0.000: (MOVE ROOM1 ROOM2)
1.000: (PICK BOX ROOM2)
2.000: (PLACE BOX ROOM2)
```

Este plan indica que el robot se desplaza, recoge el objeto y lo deja en la ubicación objetivo.

5. Actividad práctica

- Adapta el dominio y problema anteriores para incluir una tercera sala intermedia y una condición de energía limitada.
- Incluye una acción de recarga para que el robot pueda continuar si la batería es baja.
- Muestra el plan resultante y justifica las decisiones tomadas en el modelado.
- graba un vídeo (2–4 minutos) mostrando cómo se lanza el sistema y cómo se genera el plan. Incluye el enlace en el README del repositorio.